

计算机组成原理创新实验

实验名称：8 位 AI 运算器设计-第三周

学号：2410668

姓名：吕昊远

班次：张金、董前琨

一、实验任务

在第三周，我们将完成对于模块的改进，主要实现在 LUT 资源占用和计算时延这两方面的优化。通过咨询大模型以及查询相关的资料，最终选择从以下几个方面进行优化，接下来将详细讲述具体的实现。

1. 进位链加法器

首先，可以将原来的加法器改为进位链加法器，把 adder_8 替换为显式实例化的 CARRY4 原语，强制综合器使用专用进位链资源而非 LUT。其中第一级 CARRY4 负责处理低位 sum[3:0]，CO[3] 级联给第二级；第二级 CARRY4 则是处理高位 sum[7:4]，CO[3] 为溢出进位。

具体的模块代码如下：

```
1 module adder_8_carry (
2     input  wire [7:0] a,
3     input  wire [7:0] b,
4     output wire [7:0] sum,
5     output wire      cout    // 进位
6 );
7     // S = a XOR b, 为 CARRY4 的传播输入
8     wire [7:0] s = a ^ b;
9
10    // 第一级 CARRY4：处理低4位 [3:0]
11    wire [3:0] co_low;
12    CARRY4 u_carry_low (
13        .CYINIT (1'b0 ),    // 初始进位为0
14        .CI      (1'b0 ),    // 级联进位输入（第一级固定为0）
15        .DI      (a[3:0] ),   // 生成输入 = a
16        .S       (s[3:0] ),   // 传播输入 = a^b
17        .O       (sum[3:0]),   // 和输出
18        .CO      (co_low )    // 进位输出，CO[3]传给下一级
19    );
```

```

20
21 // 第二级 CARRY4：处理高4位 [7:4]
22 wire [3:0] co_high;
23 CARRY4 u_carry_high (
24     .CYINIT (1'b0      ),
25     .CI      (co_low[3] ), // 低4位的进位输出级联
26     .DI      (a[7:4]    ),
27     .S      (s[7:4]    ),
28     .O      (sum[7:4]   ),
29     .CO      (co_high   )
30 );
31
32 assign cout = co_high[3]; // 最终进位
33 endmodule

```

2. 加法树并行累加

原始的串行相加可以改为树形结构进行并行运算来减少运算时延，同时，在卷积核输出后直接连接 relu 激活层，这里可以把这两部放在同一个模块里，直接输出激活后数据。

具体的模块代码如下：

```

1 module conv3x3_tree_unit(
2     input wire [71:0] pixels,
3     input wire [71:0] weights,
4     output wire [7:0] conv_out
5 );
6     wire [7:0] p[0:8], w[0:8], m[0:8];
7     genvar i;
8     generate
9         for (i = 0; i < 9; i = i + 1) begin : MULT_GEN
10             assign p[i] = pixels[i*8 +: 8];
11             assign w[i] = weights[i*8 +: 8];
12             // 乘法操作
13             mult_8_dsp u_m (.a(p[i]), .b(w[i]), .p(m[i]));
14         end
15     endgenerate
16
17     // 加法树实现并行累加
18     wire [7:0] s1_0, s1_1, s1_2, s1_3;
19     wire [7:0] s2_0, s2_1;
20     wire [7:0] s3_0;
21     wire [7:0] sum;
22
23     adder_8_carry u_add1 (.a(m[0]), .b(m[1]), .sum(s1_0));
24     adder_8_carry u_add2 (.a(m[2]), .b(m[3]), .sum(s1_1));
25     adder_8_carry u_add3 (.a(m[4]), .b(m[5]), .sum(s1_2));
26     adder_8_carry u_add4 (.a(m[6]), .b(m[7]), .sum(s1_3));
27
28     adder_8_carry u_add5 (.a(s1_0), .b(s1_1), .sum(s2_0));
29     adder_8_carry u_add6 (.a(s1_2), .b(s1_3), .sum(s2_1));

```

```

30
31     adder_8_carry u_add7 (.a(s2_0), .b(s2_1), .sum(s3_0));
32     adder_8_carry u_add8 (.a(s3_0), .b(m[8]), .sum(sum));
33
34     // relu 输出
35     assign conv_out = (sum > 255) ? 8'd255 : sum[7:0]; // 截断防止溢出
36 endmodule

```

3. 移位运算替换除法

在池化层的模块中，可以直接将除 4 替换为右移两位，进一步减少 LUT 消耗。

```

1 module div_4_shift (
2     input  wire [9:0] in_data,
3     output wire [7:0] out_data
4 );
5     // 右移2位等效于除以4，减少LUT逻辑资源
6     assign out_data = (in_data >> 2);
7 endmodule

```

4. DSP 乘法器

经过上述优化后，测试发现时延确实有所减少，但是 LUT 资源并未有明显下降，考虑到可能软件对于这些自带一定的优化，自动推断使用了 CARRY4。为了对 LUT 进行有效的优化，这里改为采用 DSP 来实现乘法模块。

DSP，即 Digital Signal Processor（数字信号处理器），在 FPGA 中指的是芯片内部预先制造好的硬件单元，可以进行乘法运算，在 Xilinx 中通常为 DSP48。作为芯片出厂时就刻好的乘法器电路，可以直接进行使用，无需消耗大量 LUT 进行拼接组合。

```

1 module mult_8_dsp (
2     input  wire [7:0] a,
3     input  wire [7:0] b,
4     output wire [7:0] p
5 );
6     (* use_dsp = "yes" *) wire [15:0] full;
7     assign full = a * b;
8     assign p = full[7:0];
9 endmodule

```

5. 仿真波形验证

完成了上述的优化后，首先要对其正确性进行验证，编写仿真文件如下：

```

1 // w3_tb.v
2 `timescale 1ns / 1ps
3
4 module tb_week3();

```

```

5    reg [199:0] img_in;
6    reg [71:0]  weight_in;
7    wire [31:0] cnn_out;
8
9    cnn_core_top_opt u_top (
10       .img_5x5(img_in),
11       .weight_3x3(weight_in),
12       .out_2x2(cnn_out)
13   );
14
15   initial begin
16       // Test 1: 均匀图像
17       img_in = {25{8'd10}};
18       weight_in = {9{8'd1}};
19       #100;
20       $display("Test1 | weight: %h | img: %h | out: %d, %d, %d, %d",
21               weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
22               cnn_out[7:0]);
23
24       // Test 2: 孤点
25       img_in = 200'b0;
26       img_in[7:0] = 8'd40; // 左上角第一个点为40
27       weight_in = {9{8'd1}};
28       #100;
29       $display("Test2 | weight: %h | img: %h | out: %d, %d, %d, %d",
30               weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
31               cnn_out[7:0]);
32
33       // Test 3: 权重
34       img_in = {25{8'd10}};
35       weight_in = {9{8'd2}}; // 权重变为2
36       #100;
37       $display("Test3 | weight: %h | img: %h | out: %d, %d, %d, %d",
38               weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
39               cnn_out[7:0]);
40
41       // Test 4: 对角线
42       img_in = 200'b0;
43       img_in[7:0] = 8'd10;
44       img_in[55:48] = 8'd10;
45       img_in[103:96] = 8'd10;
46       img_in[151:144] = 8'd10;
47       img_in[199:192] = 8'd10;
48       weight_in = {9{8'd1}};
49       #100;
50       $display("Test4 | weight: %h | img: %h | out: %d, %d, %d, %d",
51               weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
52               cnn_out[7:0]);
53
54       $finish;

```

```
end
endmodule
```

得到仿真波形如图：

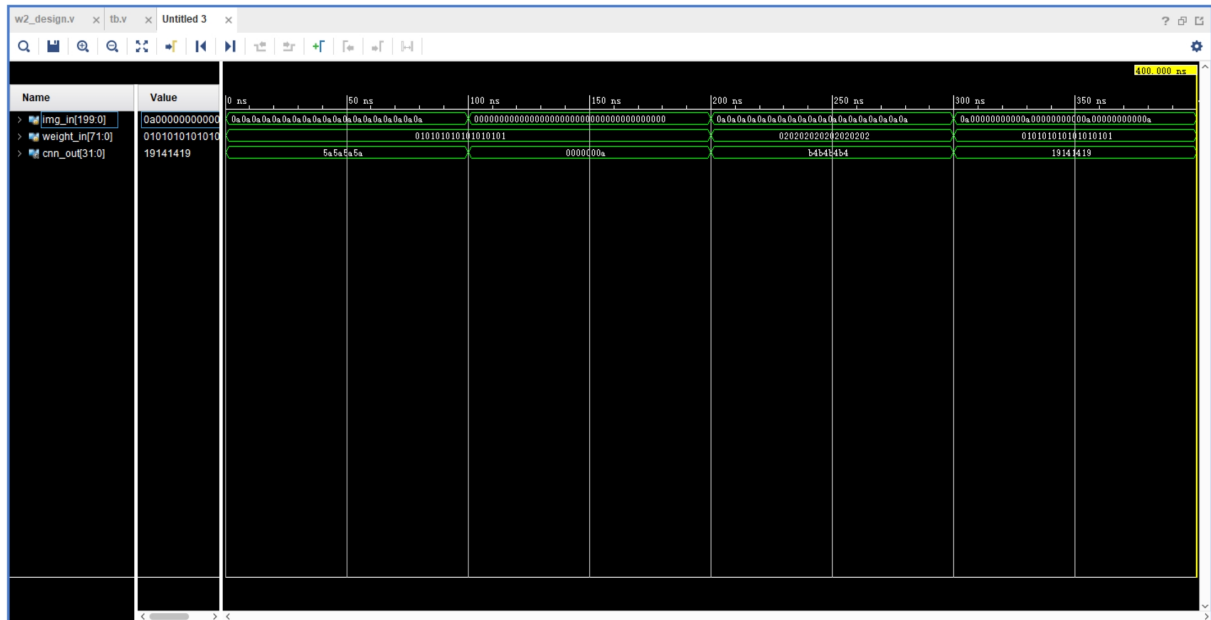


图 1: 仿真结果

为了展示更详细的测试数据，在代码中使用 `display` 进行打印，将设置的图像矩阵和权重矩阵以及最终计算得出的特征矩阵结果输出至控制台。

控制台输出结果如下:

[illegible][illegible][illegible]

```
Test4 | weight: 0101010101010101 | img: 0a000000000000a00000000000a0000000000a  
| out: 25, 20, 20, 25
```

为方便观察，改为矩阵的形式表示：

Test 1

卷积核权重矩阵 (3×3):

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

输入图像矩阵 (5×5):

$$\begin{pmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 90 & 90 \\ 90 & 90 \end{pmatrix}$$

Test 2

卷积核权重矩阵 (3×3):

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

输入图像矩阵 (5×5):

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 40 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 0 & 0 \\ 0 & 10 \end{pmatrix}$$

Test 3

卷积核权重矩阵 (3×3):

$$\begin{pmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \\ 2 & 2 & 2 \end{pmatrix}$$

输入图像矩阵 (5×5):

$$\begin{pmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 180 & 180 \\ 180 & 180 \end{pmatrix}$$

Test 4

卷积核权重矩阵 (3×3):

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

输入图像矩阵 (5×5):

$$\begin{pmatrix} 10 & 0 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 \\ 0 & 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 0 & 10 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 25 & 20 \\ 20 & 25 \end{pmatrix}$$

经计算，上述几组测试数据结果完全正确，模块代码的计算逻辑准确无误。接下来将与第二周的模块进行对比，分析其优化效果。

二、对比分析

依次运行 Synthesis 和 Implementation，点击 Synthesis 中的 Report Utilization 查看报告，再点击 Implementation 中的 Report Timing Summary 查看时序分析，分别得到以下截图：

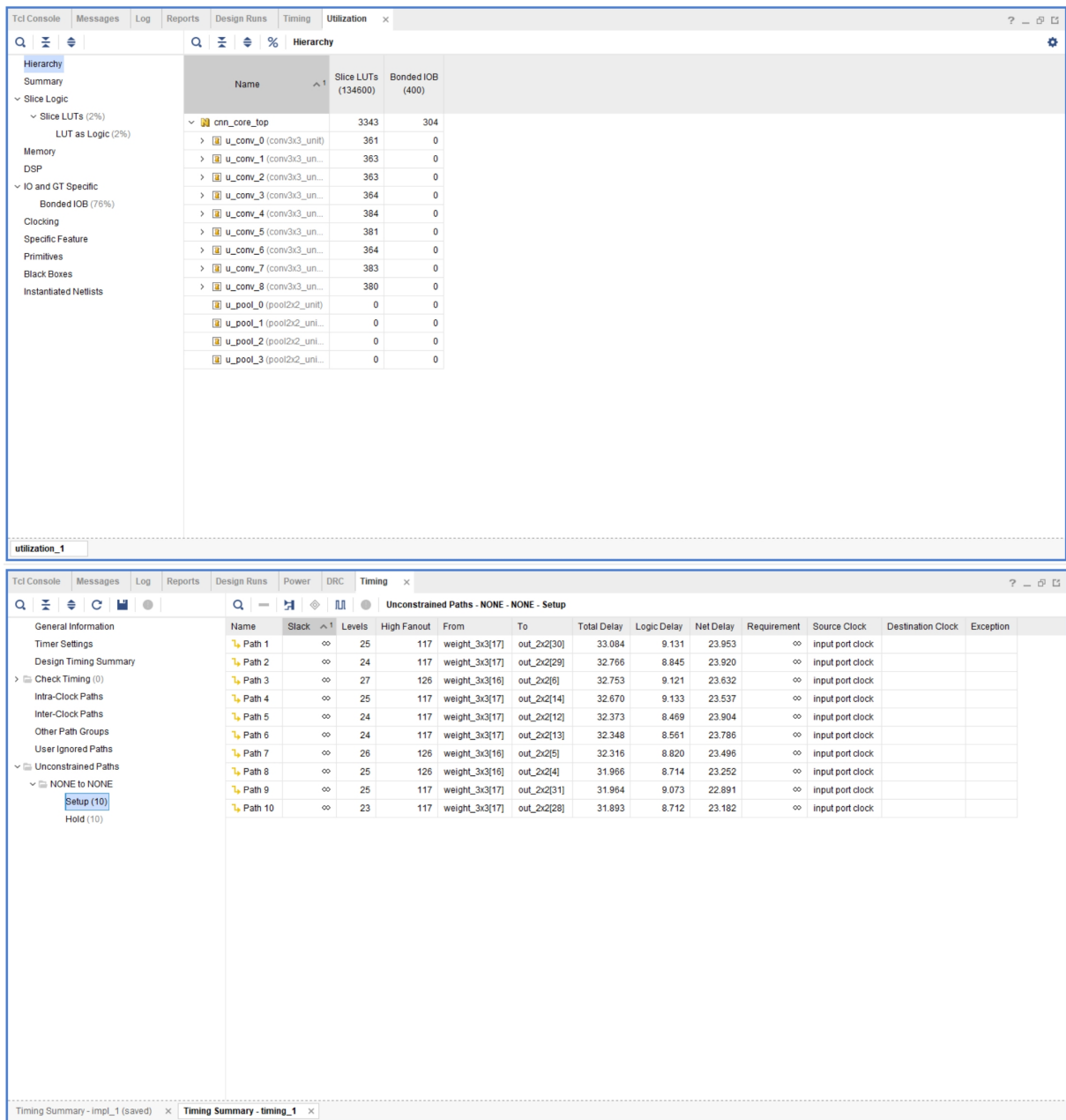


图 2: 第二周代码测试结果

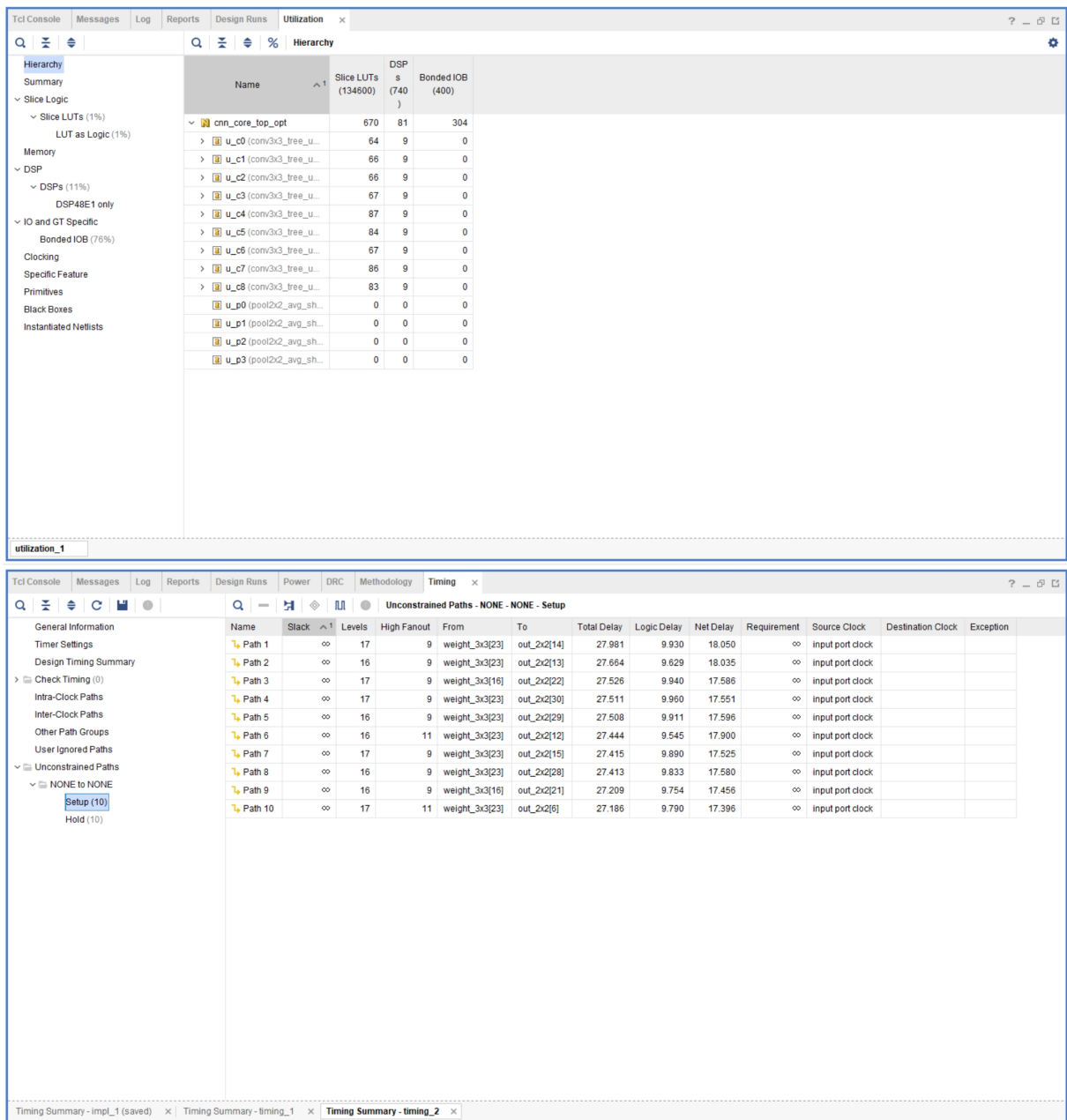


图 3: 第三周代码测试结果

将上述结果整理为表格：

表 1: 优化前后 LUT 与时序对比		
对比项	Week2	Week3
LUT 占用	3343	670
DSPs	0	81
最大时延	33.084 ns	27.981 ns
逻辑延迟	9.131 ns	9.930 ns
布线延迟	23.953 ns	18.050 ns
逻辑级数	25	17

由表可知，在引入如上各种优化后，电路资源与时序性能均得到显著改善。在资源方面，Slice LUTs 占用量从 3343 降至 670，降幅达 80%，81 个乘法器运算全部转移至片上 DSP48E1 硬核，彻底消除了乘法器对逻辑资源的占用；在时序方面，最大路径时延从 33.084ns 降至 27.981ns，其中布线延迟从 23.953ns 缩短至 18.050ns，是时序改善的主要来源，这是由于 DSP48 硬核在芯片上位置固定集中，相比分散的 LUT 乘法器显著缩短了信号走线长度。而逻辑延迟从 9.131ns 小幅上升至 9.930ns，则是因为 DSP48E1 自身固有的传播延迟引入的代价，与其他的优化有所抵消。综合来看，使用 DSP 是本设计中资源优化的核心改进，在大幅降低 LUT 占用的同时兼顾了时序性能的改善。

附录

优化后的完整模块代码：

```
1  `timescale 1ns / 1ps
2
3  module mult_8_dsp (
4      input  wire [7:0] a,
5      input  wire [7:0] b,
6      output wire [7:0] p
7  );
8      (* use_dsp = "yes" *) wire [15:0] full;
9      assign full = a * b;
10     assign p = full[7:0];
11 endmodule
12
13 module adder_8_carry (
14     input  wire [7:0] a,
15     input  wire [7:0] b,
16     output wire [7:0] sum,
17     output wire      cout    // 进位
18 );
19     // S = a XOR b, 为 CARRY4 的传播输入
20     wire [7:0] s = a ^ b;
21
22     // 第一级 CARRY4: 处理低4位 [3:0]
```

```

23     wire [3:0] co_low;
24     CARRY4 u_carry_low (
25         .CYINIT (1'b0 ),    // 初始进位为0
26         .CI      (1'b0 ),    // 级联进位输入 (第一级固定为0)
27         .DI      (a[3:0] ),  // 生成输入 = a
28         .S       (s[3:0] ),  // 传播输入 = a^b
29         .O       (sum[3:0]),  // 和输出
30         .CO      (co_low )   // 进位输出, CO[3]传给下一级
31     );
32
33     // 第二级 CARRY4: 处理高4位 [7:4]
34     wire [3:0] co_high;
35     CARRY4 u_carry_high (
36         .CYINIT (1'b0 ),
37         .CI      (co_low[3] ), // 低4位的进位输出级联至此
38         .DI      (a[7:4] ),
39         .S       (s[7:4] ),
40         .O       (sum[7:4] ),
41         .CO      (co_high )
42     );
43
44     assign cout = co_high[3]; // 最终进位
45 endmodule
46
47 // 加法树
48 module conv3x3_tree_unit(
49     input wire [71:0] pixels,
50     input wire [71:0] weights,
51     output wire [7:0] conv_out
52 );
53     wire [7:0] p[0:8], w[0:8], m[0:8];
54     genvar i;
55     generate
56         for (i = 0; i < 9; i = i + 1) begin : MULT_GEN
57             assign p[i] = pixels[i*8 +: 8];
58             assign w[i] = weights[i*8 +: 8];
59             // 乘法操作
60             mult_8_dsp u_m (.a(p[i]), .b(w[i]), .p(m[i]));
61         end
62     endgenerate
63
64     // 加法树实现并行累加
65     wire [7:0] s1_0, s1_1, s1_2, s1_3;
66     wire [7:0] s2_0, s2_1;
67     wire [7:0] s3_0;
68     wire [7:0] sum;
69
70     adder_8_carry u_add1 (.a(m[0]), .b(m[1]), .sum(s1_0));
71     adder_8_carry u_add2 (.a(m[2]), .b(m[3]), .sum(s1_1));
72     adder_8_carry u_add3 (.a(m[4]), .b(m[5]), .sum(s1_2));

```

```

73     adder_8_carry u_add4 (.a(m[6]), .b(m[7]), .sum(s1_3));
74
75     adder_8_carry u_add5 (.a(s1_0), .b(s1_1), .sum(s2_0));
76     adder_8_carry u_add6 (.a(s1_2), .b(s1_3), .sum(s2_1));
77
78     adder_8_carry u_add7 (.a(s2_0), .b(s2_1), .sum(s3_0));
79     adder_8_carry u_add8 (.a(s3_0), .b(m[8]), .sum(sum));
80
81     // relu输出
82     assign conv_out = (sum > 255) ? 8'd255 : sum[7:0]; // 截断防止溢出
83 endmodule
84
85 // 使用移位运算替换除法
86 module div_4_shift (
87     input  wire [9:0] in_data,
88     output wire [7:0] out_data
89 );
90     // 右移2位等效于除以4，减少LUT逻辑资源
91     assign out_data = (in_data >> 2);
92 endmodule
93
94 module pool2x2_avg_shift(
95     input  wire [7:0] p1, p2, p3, p4,
96     output wire [7:0] pool_out
97 );
98     wire [9:0] sum = p1 + p2 + p3 + p4;
99
100     div_4_shift u_div (.in_data(sum), .out_data(pool_out));
101 endmodule
102
103 // 顶层模块
104 module cnn_core_top_opt(
105     input  wire [199:0] img_5x5,
106     input  wire [71:0]  weight_3x3,
107     output wire [31:0]  out_2x2
108 );
109     wire [7:0] img[0:24];
110     genvar k;
111     generate
112         for (k = 0; k < 25; k = k + 1) begin : IMG_UNPACK
113             assign img[k] = img_5x5[k*8 +: 8];
114         end
115     endgenerate
116
117     wire [7:0] c[0:8];
118     // 卷积
119     conv3x3_tree_unit u_c0({img[12],img[11],img[10],img[7],img[6],img[5],img[2],img
        [1],img[0]}, weight_3x3, c[0]);
120     conv3x3_tree_unit u_c1({img[13],img[12],img[11],img[8],img[7],img[6],img[3],img
        [2],img[1]}, weight_3x3, c[1]);

```

```

121 conv3x3_tree_unit u_c2({img[14],img[13],img[12],img[9],img[8],img[7],img[4],img
    [3],img[2]}, weight_3x3, c[2]);
122 conv3x3_tree_unit u_c3({img[17],img[16],img[15],img[12],img[11],img[10],img[7],
    img[6],img[5]}, weight_3x3, c[3]);
123 conv3x3_tree_unit u_c4({img[18],img[17],img[16],img[13],img[12],img[11],img[8],
    img[7],img[6]}, weight_3x3, c[4]);
124 conv3x3_tree_unit u_c5({img[19],img[18],img[17],img[14],img[13],img[12],img[9],
    img[8],img[7]}, weight_3x3, c[5]);
125 conv3x3_tree_unit u_c6({img[22],img[21],img[20],img[17],img[16],img[15],img
    [12],img[11],img[10]}, weight_3x3, c[6]);
126 conv3x3_tree_unit u_c7({img[23],img[22],img[21],img[18],img[17],img[16],img
    [13],img[12],img[11]}, weight_3x3, c[7]);
127 conv3x3_tree_unit u_c8({img[24],img[23],img[22],img[19],img[18],img[17],img
    [14],img[13],img[12]}, weight_3x3, c[8]);
128
129 // 池化
130 wire [7:0] f[0:3];
131 pool2x2_avg_shift u_p0(c[0], c[1], c[3], c[4], f[0]);
132 pool2x2_avg_shift u_p1(c[1], c[2], c[4], c[5], f[1]);
133 pool2x2_avg_shift u_p2(c[3], c[4], c[6], c[7], f[2]);
134 pool2x2_avg_shift u_p3(c[4], c[5], c[7], c[8], f[3]);
135
136 assign out_2x2 = {f[3], f[2], f[1], f[0]};
137 endmodule

```